AI ASSISTED CODING

Lab Assignment-20.5

Name : T.Shylasri

H.T.NO: 2303A51876

Batch-14(LAB-20)

Date: 26-03-2026

Lab 20 – Security Testing: Identifying Vulnerabilities in AI-Generated Code

Lab Objectives:

• Understand how to test AI-generated code for common security vulnerabilities.

• Learn to apply secure coding principles while analyzing AI outputs.

• Practice detecting risks such as SQL injection, XSS, hardcoded credentials, and weak encryption.

• Enhance code reliability and safety by using AI for secure refactoring.

Task 1 – Password Storage Vulnerability Check

Task:

Analyze an AI-generated signup script for insecure password storage.

Instructions:

• Prompt AI to generate a Python user registration program.

• Check whether passwords are stored in plain text.

• Improve the script using hashing (bcrypt or hashlib).

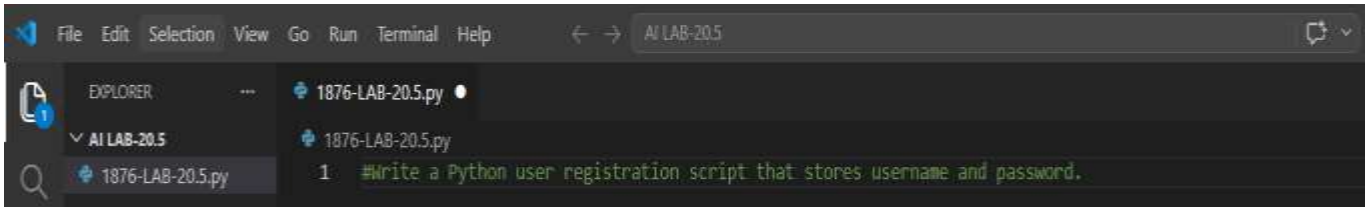
Expected Output:

• A secure registration script with hashed password storage.

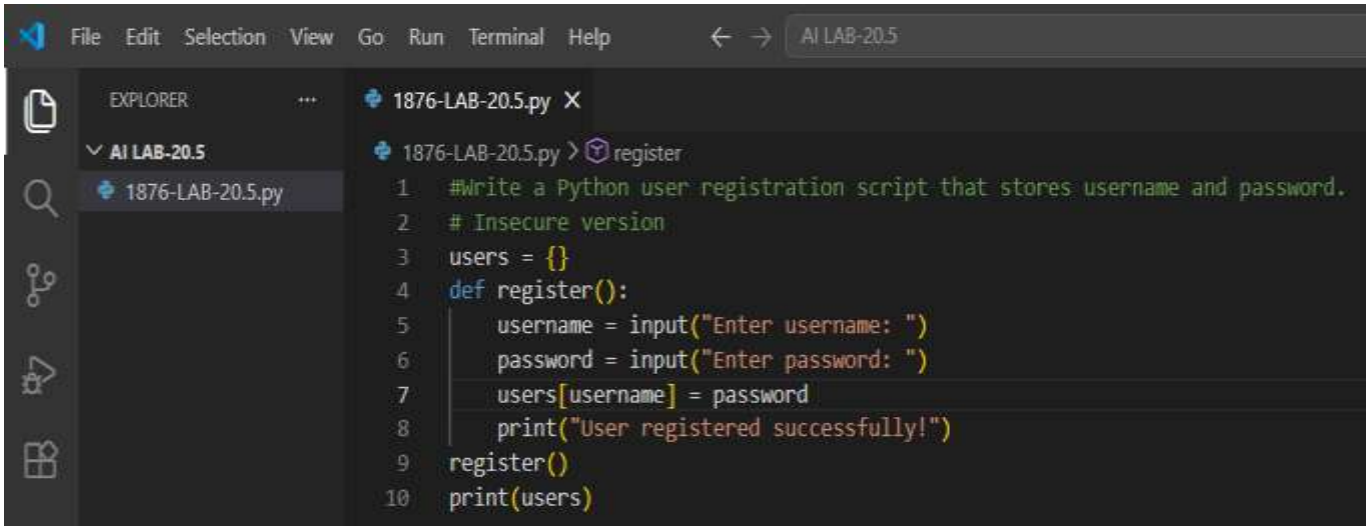
AI Prompt

#Write a Python user registration script that stores username and password.

Prompt Screenshot



AI-Generated Insecure Code



Output

Vulnerability Identified

- Passwords are stored in **plain text**
- Anyone accessing the system can read all passwords
- High risk of data breach

Secure Code (Improved Version Using bcrypt)

```python
#Write a Python user registration script that stores username and password.
# Secure version
import bcrypt
users = {}
def register():
    username = input("Enter username: ")
    password = input("Enter password: ").encode('utf-8')
    hashed_password = bcrypt.hashpw(password, bcrypt.gensalt())
    users[username] = hashed_password
    print("User registered securely!")
def login():
    username = input("Enter username: ")
    password = input("Enter password: ").encode('utf-8')
    if username in users:
        if bcrypt.checkpw(password, users[username]):
            print("Login successful!")
        else:
            print("Invalid password!")
    else:
        print("User not found!")
register()
login()
```

Output

```
PS C:\Users\91986\OneDrive\Documents\AI LAB-20.5> & "C:/Program Files/Python313/python.exe" "c:/Users/91986/OneDrive/Documents/AI LAB-20.5/1876-LAB-20.5.py"
Enter username: Thulasi Shylasri
Enter password: MyPassword@123
User registered securely!
```

Observation

- In the insecure version, passwords are directly visible
- In the secure version:
    - Passwords are hashed
    - Stored values are unreadable
    - Authentication still works correctly

Explanation

- **Plain-text storage** exposes user credentials
- **bcrypt hashing**:
    - Converts password into a secure hash
    - Adds salt automatically
    - Makes reverse-engineering extremely difficult

Justification

- Using hashing ensures:
  - Data confidentiality
  - Protection against database leaks
  - Compliance with security best practices
- bcrypt is preferred because:
  - Built-in salting
  - Resistant to brute-force attacks
  - Industry standard

Conclusion

- AI-generated code may contain **critical security flaws**
- Plain-text password storage is a **serious vulnerability**
- Implementing hashing (like bcrypt) significantly improves security
- Security testing is essential when using AI-generated code

Task 2 – Hardcoded Secrets Detection

Task:

Evaluate AI-generated code for hardcoded API keys or credentials.

Instructions:

• Ask AI to generate a script that connects to an external API.

• Identify if API keys/passwords are written directly in code.

• Refactor using environment variables or config files.

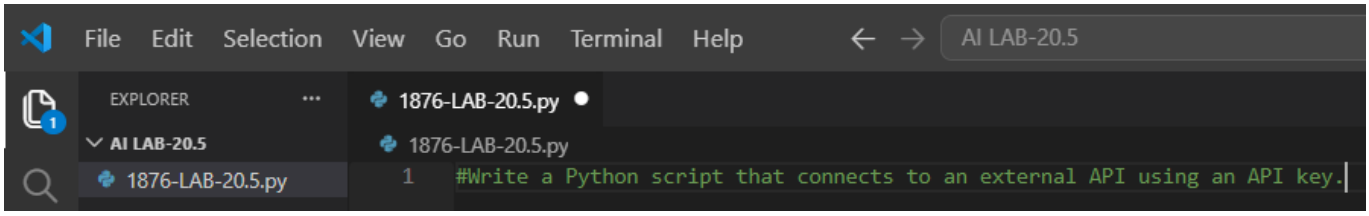
Expected Output:

• A secure script with secrets managed properly.

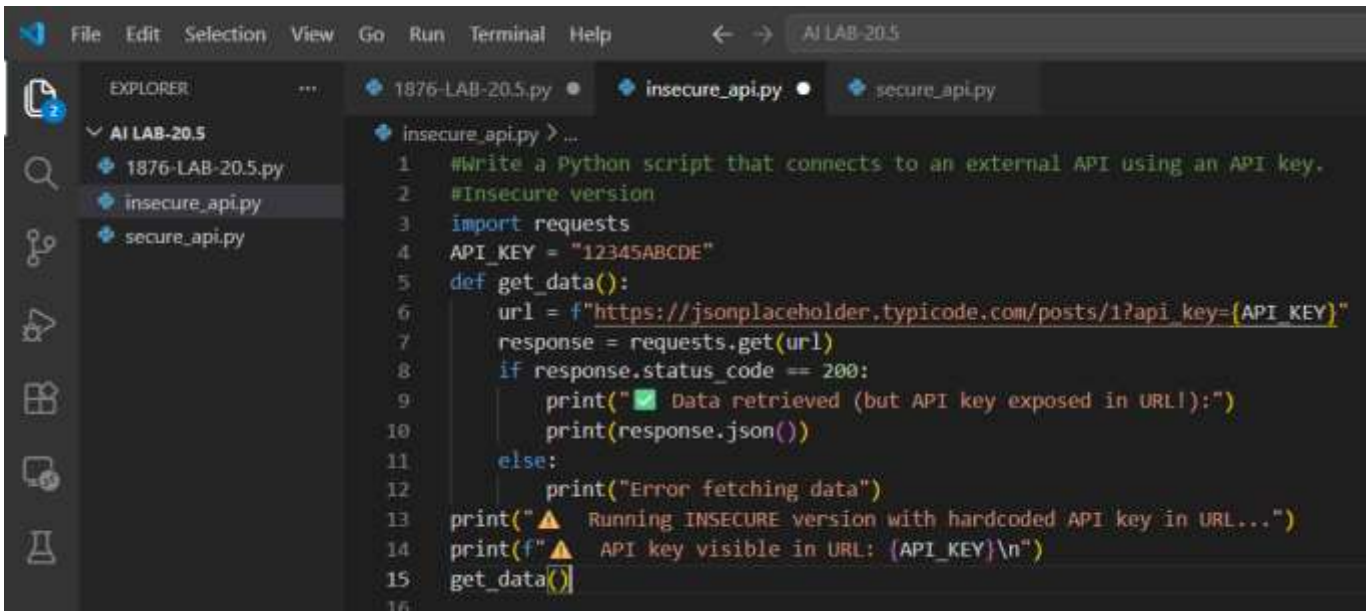
Prompt

#Write a Python script that connects to an external API using an API key.

Screenshot



AI-Generated Insecure Code



Output

Vulnerability Identified

- API key is **hardcoded directly in the source code**
- Risks:
  - Exposure if code is shared (GitHub, etc.)
  - Unauthorized API usage
  - Difficult to rotate/change keys

Secure Code (Using Environment Variables)

```python
#Write a Python script that connects to an external API using an API key.
#SECURE VERSION
import requests
import os
def get_data():
    API_KEY = os.getenv("API_KEY")  # ✅ Secure retrieval
    if not API_KEY:
        print("API key not found. Set environment variable.")
        return
    url = f"https://api.example.com/data?api_key={API_KEY}"
    response = requests.get(url)

    if response.status_code == 200:
        print(response.json())
    else:
        print("Error fetching data")
get_data()
```

Output

```
For Windows:
   set API_KEY=12345ABCDE

For Linux/Mac:
   export API_KEY=12345ABCDE
```

Observation

- In the insecure version:
    - API key is visible in code
- In the secure version:
    - API key is hidden from source code
    - Retrieved dynamically at runtime

Explanation

- Hardcoding secrets makes them easy to leak
- Environment variables:
    - Store sensitive data outside the code
    - Allow safe configuration across environments
- Improves security without changing program logic

Justification

- Prevents accidental exposure in:
  - Version control systems
  - Shared codebases
- Enables:
  - Easy key rotation
  - Secure deployment practices
- Follows industry standards (DevSecOps practices)

Conclusion

- AI-generated code may include **hardcoded sensitive data**
- This is a **critical security vulnerability**
- Using environment variables ensures:
  - Better security
  - Flexibility
  - Compliance with best practices

Task 3 – File Upload Security Validation

Task:

Test an AI-generated file upload program for unsafe file handling.

Instructions:

• Prompt AI to generate a Flask file upload example.

• Check for vulnerabilities like uploading executable files.

• Add validation for file type, size, and secure filename handling.

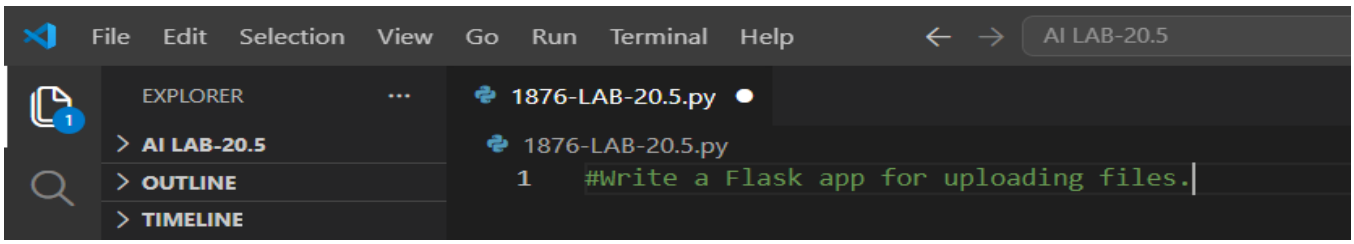
Expected Output:

• A secure file upload implementation.

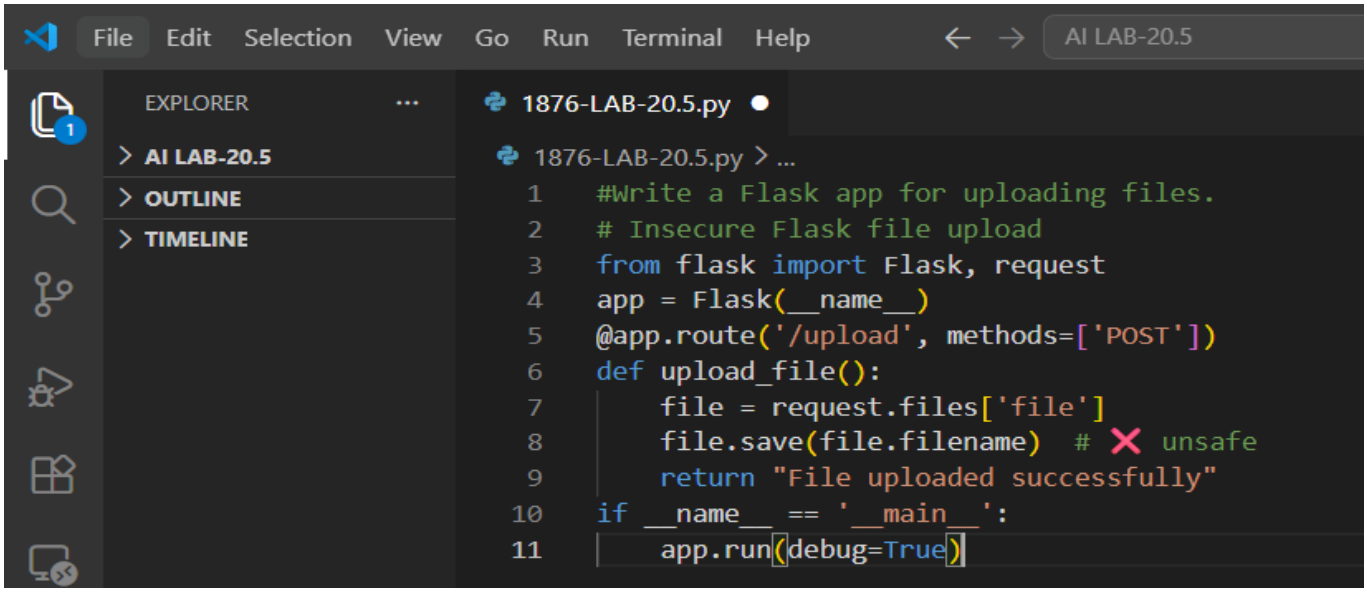
Prompt

#Write a Flask app for uploading files.

Prompt Screenshot



AI-Generated Insecure Code



Output

Vulnerabilities Identified

- No file type validation
- No file size restriction
- Unsafe filename usage (file.filename)
- Allows uploading:
  - .exe, .sh, .py (malicious files)
- Risk of:
  - Remote Code Execution (RCE)
  - Server compromise

Secure Code (Improved Version)

```python
#Write a Flask app for uploading files.
#Secured Version
from flask import Flask, request
from werkzeug.utils import secure_filename
import os
app = Flask(__name__)
UPLOAD_FOLDER = "uploads"
ALLOWED_EXTENSIONS = {"txt", "pdf", "png", "jpg", "jpeg"}
MAX_FILE_SIZE = 2 * 1024 * 1024  # 2 MB
app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER
if not os.path.exists(UPLOAD_FOLDER):
    os.makedirs(UPLOAD_FOLDER)
def allowed_file(filename):
    return "." in filename and filename.rsplit(".", 1)[1].lower() in ALLOWED_EXTENSIONS
@app.route('/upload', methods=['POST'])
def upload_file():
    file = request.files.get('file')
    if not file:
        return "No file uploaded"

    if not allowed_file(file.filename):
        return "Invalid file type"

    file.seek(0, 2)  # Seek to end
    file_size = file.tell()
    file.seek(0)  # Back to start
    if file_size > MAX_FILE_SIZE:
        return "File too large"

    filename = secure_filename(file.filename)
    file.save(os.path.join(app.config["UPLOAD_FOLDER"], filename))

    return "File uploaded securely"

if __name__ == '__main__':
    app.run(debug=True)
```

Output

```
PS C:\Users\91986\OneDrive\Documents\AI LAB-20.5> & "C:/Program Files/Python313/python.exe" "c:/Users/91986/OneDrive/Documents/AI LAB-20.5/1876-LAB-20.5.py"
 * Serving Flask app '1876-LAB-20.5'
 * Debug mode: on
```

Observation

- In insecure version:
  - Any file can be uploaded
  - Filename is not sanitized
- In secure version:
  - Only allowed file types accepted
  - File size is restricted
  - Filenames are cleaned using secure_filename()

Explanation

- **Unrestricted uploads** can allow attackers to:
  - Upload malicious scripts
  - Execute code on server
- Security measures used:
  - File type whitelist
  - File size limit
  - Secure filename handling
- secure_filename() prevents:
  - Path traversal (../../etc/passwd)
  - Injection attacks

Justification

- Protects server from malicious files
- Prevents unauthorized execution
- Ensures safe file storage
- Follows secure web development practices

Conclusion

- AI-generated file upload code may lack proper validation
- Unchecked uploads are a **serious security risk**
- Adding validation ensures:
  - Application safety
  - Data integrity
  - Secure user interaction

Task 4 – Command Injection Prevention

Task:

Analyze an AI-generated Python script that executes system commands.

Instructions:

- Ask AI to generate code using os.system() or subprocess.
- Identify command injection risks with user input.
- Fix using safe subprocess calls and input validation.

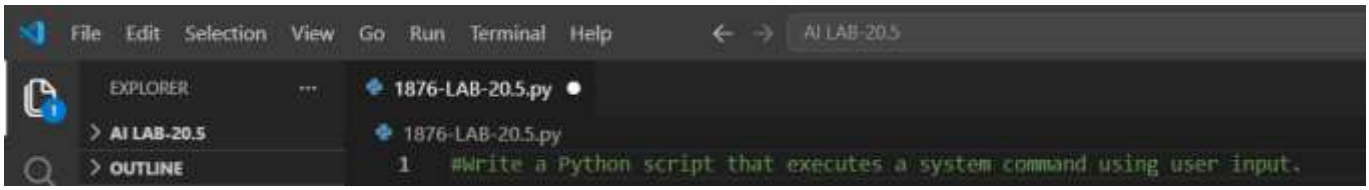
Expected Output:

- A secure command execution script.

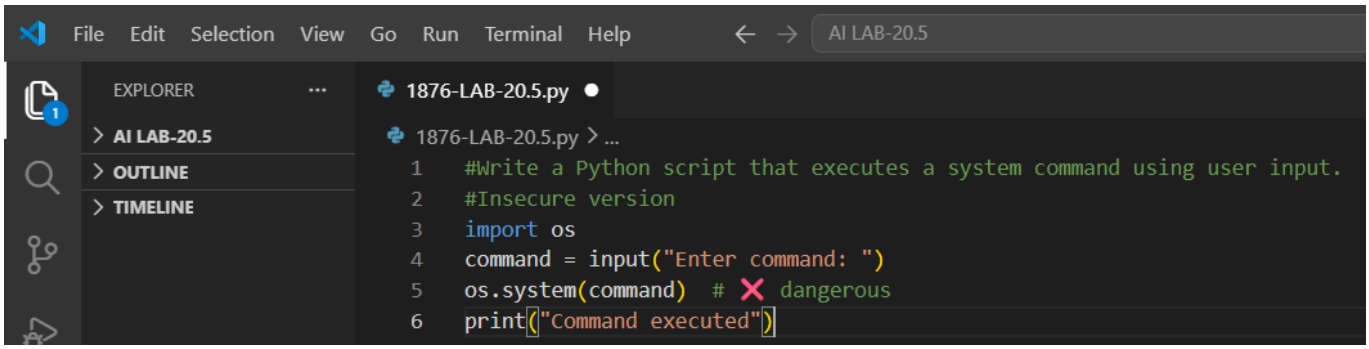
Prompt

#Write a Python script that executes a system command using user input.

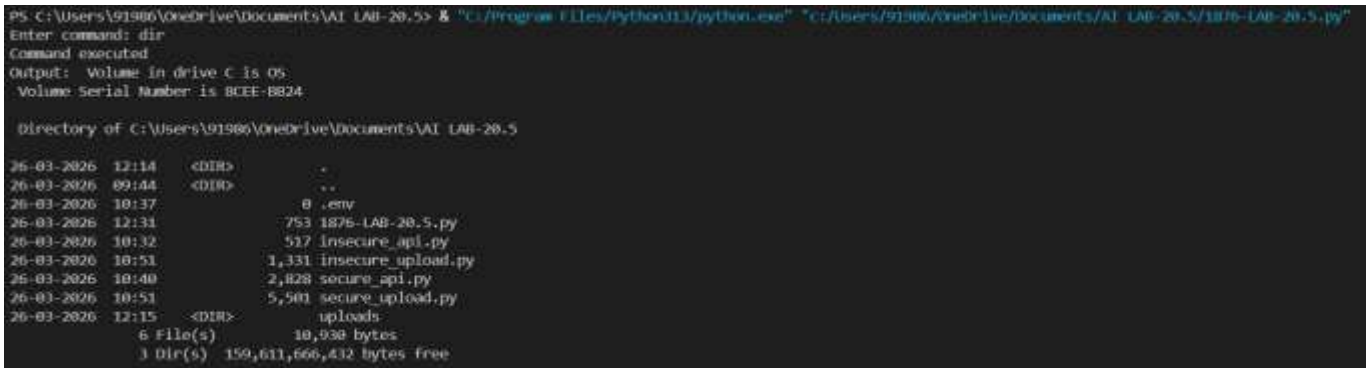
Prompt Screenshot



AI-Generated Insecure Code



Output



Vulnerability Identified

- Direct execution of user input as system command
- Allows attackers to run:
  - rm -rf / (delete files)
  - dir && malicious_command
- Leads to:
  - Command Injection
  - Full system compromise

Secure Code (Improved Version)

```python
#Write a Python script that executes a system command using user input.
# Secure version
import subprocess
import shlex
# Whitelist of allowed commands (Windows)
ALLOWED_COMMANDS = {'dir', 'echo', 'date', 'whoami', 'ver'}
command_input = input("Enter command: ")
try:
    args = shlex.split(command_input)
    if not args:
        print("No command entered")
    elif args[0] not in ALLOWED_COMMANDS:
        print("Command not allowed")
    else:
        result = subprocess.run(['cmd', '/c'] + args, capture_output=True, text=True)
        print("Command executed")
        print("Output:", result.stdout)
        if result.stderr:
            print("Error:", result.stderr)
except ValueError:
    print("Invalid command syntax")
```

Output

```
les/Python313/python.exe" "c:/Users/91986/OneDrive/Documents/AI LAB-20.5/1876-LAB-20.5.py"
Enter command: dir
Command executed
Output:  Volume in drive C is OS
 Volume Serial Number is 8CEE-8B24

 Directory of C:\Users\91986\OneDrive\Documents\AI LAB-20.5

26-03-2026  12:14    <DIR>          .
26-03-2026  09:44    <DIR>          ..
26-03-2026  10:37                0 .env
26-03-2026  12:31              753 1876-LAB-20.5.py
26-03-2026  10:32              517 insecure_api.py
Enter command: dir
Command executed
Output:  Volume in drive C is OS
 Volume Serial Number is 8CEE-8B24

 Directory of C:\Users\91986\OneDrive\Documents\AI LAB-20.5

26-03-2026  12:14    <DIR>          .
26-03-2026  09:44    <DIR>          ..
26-03-2026  10:37                0 .env
26-03-2026  12:31              753 1876-LAB-20.5.py
26-03-2026  10:32              517 insecure_api.py
 Directory of C:\Users\91986\OneDrive\Documents\AI LAB-20.5

26-03-2026  12:14    <DIR>          .
26-03-2026  09:44    <DIR>          ..
26-03-2026  10:37                0 .env
26-03-2026  12:31              753 1876-LAB-20.5.py
26-03-2026  10:32              517 insecure_api.py
26-03-2026  09:44    <DIR>          ..
26-03-2026  10:37                0 .env
26-03-2026  12:31              753 1876-LAB-20.5.py
26-03-2026  10:32              517 insecure_api.py
26-03-2026  10:37                0 .env
26-03-2026  12:31              753 1876-LAB-20.5.py
26-03-2026  10:32              517 insecure_api.py
26-03-2026  12:31              753 1876-LAB-20.5.py
26-03-2026  10:32              517 insecure_api.py
26-03-2026  10:32              517 insecure_api.py
26-03-2026  10:51            1,331 insecure_upload.py
26-03-2026  10:40            2,828 secure_api.py
26-03-2026  10:51            5,581 secure_upload.py
26-03-2026  12:15    <DIR>          uploads
               6 File(s)         10,930 bytes
               3 Dir(s)  159,618,793,984 bytes free
```

Observation

- In insecure version:
  - Any command is executed blindly
- In secure version:
  - Only predefined commands are allowed
  - No shell interpretation
  - Malicious input is blocked

Explanation

- os.system() executes commands via shell → unsafe
- Attackers can inject additional commands using:
  - ;, &&, ||
- Secure approach:
  - Use subprocess.run() with **list arguments**
  - Avoid shell=True
  - Validate user input against allowed commands

Justification

- Prevents execution of arbitrary system commands
- Reduces risk of:
  - Data loss
  - Unauthorized access
- Ensures controlled and safe command execution

Conclusion

- AI-generated scripts using os.system() are **highly vulnerable**
- Command injection is a **critical security flaw**
- Using subprocess safely with validation:
  - Protects the system
  - Ensures secure execution
- Always validate and restrict user inputs in system-level operations

Task 5 – Secure Input Handling in Web Forms

Task:

Check AI-generated web form code for improper input validation.

Instructions:

• Ask AI to generate an HTML + Flask feedback form.

• Identify missing validation and unsafe rendering.

• Fix using server-side validation and sanitization.

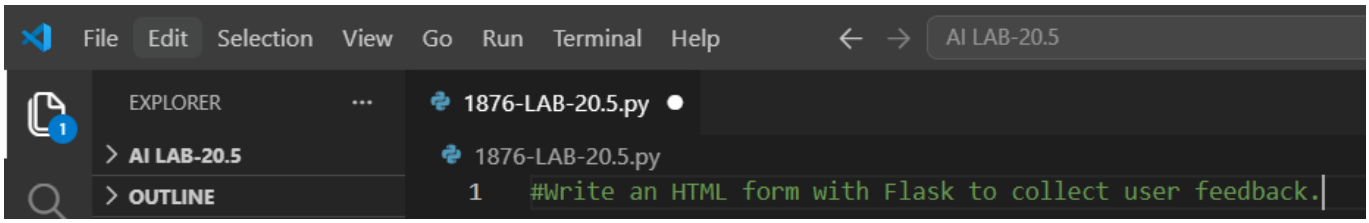
Expected Output:

• A secure form resistant to injection attacks.

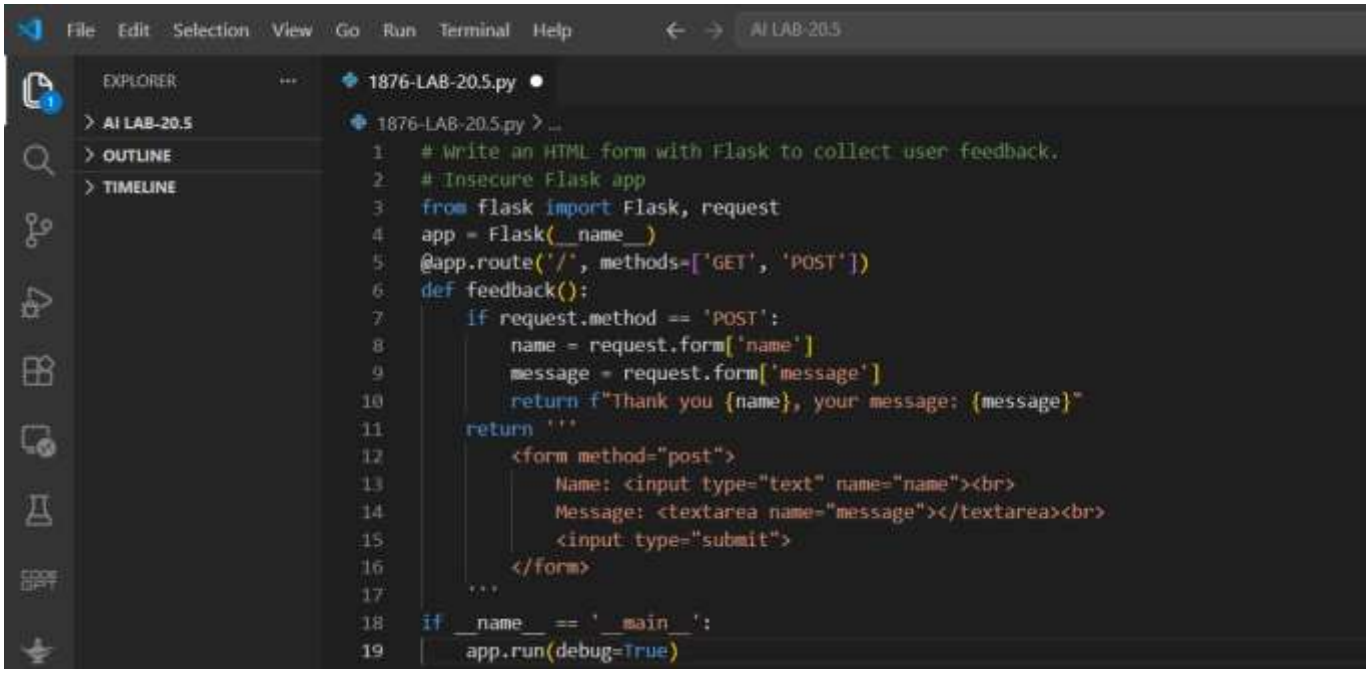
Prompt

#Write an HTML form with Flask to collect user feedback.

Prompt Screenshot



AI-Generated Insecure Code



Output

```
PS C:\Users\91986\OneDrive\Documents\AI LAB-20.5> dir


    Directory: C:\Users\91986\OneDrive\Documents\AI LAB-20.5


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
dar--l        26-03-2026     12:15                uploads
-a---l        26-03-2026     10:37              0 .env
-a---l        26-03-2026     12:47           1104 1876-LAB-20.5.py
-a---l        26-03-2026     10:32            517 insecure_api.py
-a---l        26-03-2026     10:51           1331 insecure_upload.py
-a---l        26-03-2026     10:40           2828 secure_api.py
-a---l        26-03-2026     10:51           5501 secure_upload.py
```

Vulnerabilities Identified

- No input validation
- User input rendered directly in response
- Vulnerable to **XSS (Cross-Site Scripting)**

Secure Code (Improved Version)

```python
# Write an HTML form with Flask to collect user feedback.
#SEcured Code
from flask import Flask, request, render_template_string
import html
app = Flask(__name__)
@app.route('/', methods=['GET', 'POST'])
def feedback():
    if request.method == 'POST':
        name = request.form.get('name', '').strip()
        message = request.form.get('message', '').strip()
        # ✅ Server-side validation
        if not name or not message:
            return "All fields are required!"
        if len(name) > 50 or len(message) > 200:
            return "Input too long!"
        safe_name = html.escape(name)
        safe_message = html.escape(message)
        return render_template_string("""
            <h3>Thank you {{name}}</h3>
            <p>Your message: {{message}}</p>
        """, name=safe_name, message=safe_message)
    return '''
        <form method="post">
            Name: <input type="text" name="name" required><br>
            Message: <textarea name="message" required></textarea><br>
            <input type="submit">
        </form>
    '''
if __name__ == '__main__':
    app.run(debug=True)
```

Output

```
PROBLEMS 10   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE

 * Serving Flask app '1876-LAB-20.5'
 * Debug mode: on
```

Observation

- In insecure version:
  - Script tags execute in browser
- In secure version:
  - Input is validated and escaped
  - No script execution

Explanation

- XSS occurs when user input is rendered without sanitization
- Secure practices:
  - Validate input length and presence
  - Escape HTML characters using html.escape()
  - Use safe template rendering

Justification

- Prevents:
  - Script injection
  - Session hijacking
  - Data theft
- Ensures safe interaction between user and server

Conclusion

- AI-generated web forms may lack proper validation
- Rendering raw input leads to **XSS vulnerabilities**
- Using validation + sanitization:
  - Protects users
  - Secures application
- Always treat user input as **untrusted data**